COMPANION TO THE CLEAN ARCHITECTURE GUIDE

The API Integration Playbook

From an easy public list to very complex APIs with the same Clean Architecture + Cubit

> **Question it answers:** "I'm given an API I have never seen. What do I change, and where?"

Part A · Read any API in two minutes: 12-question recon + complexity ladder

Levels 1–5 · Easy list → CRUD → pagination/files → polymorphic/ETag/429/GraphQL → jobs/streams

Part B · "I see X → do Y" matrix, 20-minute plans, universal rules

Part C/D · 24 interview Q&A and the honest verification record

> Levels 1–4 and the job pattern: strict analyzer 0 issues, **156 passing tests**, run live against JSONPlaceholder, httpbin and a GraphQL API. Real-time streams are analyzer-clean but **untested**; offline outbox, OAuth2 PKCE, request signing are **design recipes only**. Every section says which is which.

Flutter 3.47 · Dart 3.13 · flutter_bloc 9 · dio 5 · October 2026

Contents

The API Integration Playbook **3**
From an easy public list to very complex APIs, with the same Clean Architecture + Cubit 3
PART A: Read any API in two minutes 4
LEVEL 1: EASY. A public list, no auth, no envelope 11
LEVEL 2: SIMPLE. Detail, filters, and the five verbs 15
LEVEL 3: MEDIUM. Pagination, nested resources, files 20
LEVEL 4: HARD. Shapes, standards, throttling, caching, GraphQL 27
LEVEL 5: VERY COMPLEX 34
PART B: Cheat sheets 43
PART C: Interview questions for "integrate this API" (concise answers) 45
PART D: Verification record and known limits 47

The API Integration Playbook

From an easy public list to very complex APIs, with the same Clean Architecture + Cubit

> **Companion to the first book** (*Flutter API Integration: Clean Architecture + BLoC/Cubit*). That book teaches the architecture once. This one answers: **"I'm given an API I have never seen. What do I change, and where?"**
>
> **What is verified (read this):** * Levels 1–4 and the async-job pattern in Level 5: all code in this book for them is taken from a project that passes `flutter analyze` (strict mode, 0 issues) and **156 tests**. The same stack was also run **live** against `jsonplaceholder.typicode.com`, `httpbin.org` and `countries.trevorblades.com` (outputs are printed in the "Live proof" boxes). * Real-time streams (SSE / WebSocket / reconnect): the library code compiles under the same strict analyzer, **but its tests were not finished or run**. It is marked **UNTESTED** where it appears. * Offline-first outbox, OAuth2 PKCE, request signing, environments/versioning, background parsing: **not built or tested in this book.** They appear only as short design recipes, clearly labeled.

PART A: Read any API in two minutes

A.1 The recon checklist: ask these 12 questions BEFORE writing code

#	Question (look at 2–3 real responses, or the docs)	Why it matters / what it changes
1	**What does a list look like: bare array or an envelope?** `[...]` vs `{"data": [...], "total": ...}`	Model: `(json as List)` vs `json['data']`
2	**Where is pagination info?** Body fields? Headers (`x-total-count` , `Link`)? A cursor?	Chooses the `PageToken` (page number / cursor / next URL)
3	**Are there nulls / optional fields?** Which are really optional?	Nullable model fields vs strict required ones
4	**Is the shape always the same?** Or does a `type` field change it?	Polymorphic parsing (sealed class)
5	**Which verbs exist?** Does a write return the resource, an id, or nothing (204)?	Response model or `Future<void>`
6	**How are errors shaped?** `{"error": "..."}` , `{"message": ...}` , problem+json, field errors, or HTTP 200 with `errors` ?	`DioClient` error mapping; GraphQL trap
7	**Auth?** none / API key header / Bearer / OAuth2	Interceptor (first book §11–13)
8	**Rate limits?** `429` + `Retry-After` ?	Retry-After interceptor
9	**Is data cacheable?** `ETag` , `Cache-Control` ?	ETag interceptor / stale-while-revalidate
10	**Is the operation instant, or long-running?** `202 Accepted` + status URL?	Polling stream (jobs)
11	**Is it one request, or does a screen need several?** Which depend on which?	Parallel + dependent calls in a use case
12	**Is it one-shot or live?** SSE / WebSocket / long-poll?	Reconnecting stream + Bloc

A.2 The complexity ladder

Level	You see…	Example (verified here)	What changes vs the base architecture
1 Easy	A public `GET` returning a bare array	`GET /posts`	Model parses a list; add an **Empty** state
2 Simple	Detail by id, filters, the 5 verbs, simple validation	`GET /posts/1?userId=3` , `POST/PUT/PATCH/DELETE`	Query params; request models; **Command** Cubit; optimistic delete
3 Medium	Pagination in headers, nested resources, files	`Link` header pages, `/posts/1/comments` , multipart upload	**PageToken** + one reusable `PaginatedCubit` ; parallel use case; `UploadTask`
4 Hard	Polymorphic JSON, standard error bodies, caching, throttling, GraphQL	feed with `type` , ETag/304, `429` , countries GraphQL	Sealed models; `ProblemDetails` ; interceptors; **stale-while-revalidate**
5 Very complex	Long jobs, live streams, offline writes, OAuth2, signed requests	`202` + polling, SSE/WebSocket	Streams as use-case results; reconnect/backoff; (outbox, PKCE: recipes only)

The rule that keeps it simple: the **layers never change**. Only these things move:

```
DioClient (transport, errors)  ->  Model (shape of the JSON)  ->  PageToken/Entity (what the app needs)
        interceptors (cross-cutting)                        Cubit state shape (Query / Command / Stream)
```

A.3 The shared toolbox (built once, used by every level)

`DioClient` is the same class as in the first book, with a few additions that every harder level needs: **headers next to the body** (`getResponse`), a **cancel token**, **upload progress**, **download**, a **byte stream**, `429` /problem+json awareness. Errors still travel `DioException → AppException → Failure → state` .

`core/network/dio_client.dart`

```dart
import 'package:dio/dio.dart';

import '../errors/exceptions.dart';
import '../errors/problem_details.dart';
import 'api_response.dart';
import 'retry_after.dart';

/// Thin wrapper over Dio: one place for every HTTP verb and for
/// DioException -> AppException mapping.
class DioClient {
  const DioClient(this._dio);

  final Dio _dio;

  Future<dynamic> get(
    String path, {
    Map<String, dynamic>? query,
    CancelToken? cancelToken,
  }) async =>
      (await getResponse(path, query: query, cancelToken: cancelToken)).data;

  /// Body **and** headers (`x-total-count`, `Link`, `ETag`...).
  Future<ApiResponse> getResponse(
    String path, {
    Map<String, dynamic>? query,
    Map<String, String>? headers,
    CancelToken? cancelToken,
  }) async {
    final r = await _call(
      () => _dio.get<dynamic>(
        path,
        queryParameters: query,
        cancelToken: cancelToken,
        options: Options(headers: headers),
      ),
    );
    return ApiResponse(
      data: r.data,
      headers: r.headers,
      statusCode: r.statusCode ?? 0,
      fromCache: r.extra['fromCache'] == true,
    );
  }

  Future<dynamic> post(
    String path, {
    Object? data,
    Map<String, String>? headers,
    CancelToken? cancelToken,
    ProgressCallback? onSendProgress,
  }) async => (await _call(
    () => _dio.post<dynamic>(
      path,
      data: data,
      cancelToken: cancelToken,
      onSendProgress: onSendProgress,
      options: Options(headers: headers),
    ),
  )).data;

  Future<dynamic> put(
    String path, {
    Object? data,
```

```
    Map<String, String>? headers,
    CancelToken? cancelToken,
  }) async => (await _call(
    () => _dio.put<dynamic>(
      path,
      data: data,
      cancelToken: cancelToken,
      options: Options(headers: headers),
    ),
  )).data;

  Future<dynamic> patch(
    String path, {
    Object? data,
    Map<String, String>? headers,
    CancelToken? cancelToken,
  }) async => (await _call(
    () => _dio.patch<dynamic>(
      path,
      data: data,
      cancelToken: cancelToken,
      options: Options(headers: headers),
    ),
  )).data;

  Future<dynamic> delete(
    String path, {
    Map<String, String>? headers,
    CancelToken? cancelToken,
  }) async => (await _call(
    () => _dio.delete<dynamic>(
      path,
      cancelToken: cancelToken,
      options: Options(headers: headers),
    ),
  )).data;

  /// Streams the response to a file with progress (0..total bytes).
  Future<void> download(
    String url,
    String savePath, {
    ProgressCallback? onReceiveProgress,
    CancelToken? cancelToken,
  }) async {
    await _call(
      () => _dio.download(
        url,
        savePath,
        onReceiveProgress: onReceiveProgress,
        cancelToken: cancelToken,
      ),
    );
  }

  /// Raw byte stream for SSE / NDJSON / long-lived responses.
  Future<Stream<List<int>>> openStream(
    String url, {
    Map<String, String>? headers,
    CancelToken? cancelToken,
  }) async {
    final r = await _call(
      () => _dio.get<ResponseBody>(
        url,
        cancelToken: cancelToken,
        options: Options(responseType: ResponseType.stream, headers: headers),
      ),
    );
    return r.data!.stream;
  }

  Future<Response<T>> _call<T>(Future<Response<T>> Function() request) async {
    try {
```

```
      return await request();
    } on DioException catch (e) {
      throw _toException(e);
    }
  }

  AppException _toException(DioException e) => switch (e.type) {
    DioExceptionType.connectionTimeout ||
    DioExceptionType.sendTimeout ||
    DioExceptionType.receiveTimeout => const NetworkException(
      'Request timed out. Please try again.',
    ),
    DioExceptionType.connectionError => const NetworkException(
      'No internet connection.',
    ),
    DioExceptionType.cancel => const CancelledException('Request cancelled.'),
    DioExceptionType.badResponse => _fromResponse(e.response),
    // Dio wraps a JSON decoding error as `unknown` + FormatException.
    DioExceptionType.unknown when e.error is FormatException =>
      const ParsingException('Invalid response from server.'),
    _ => const NetworkException('Network error. Please try again.'),
  };

  AppException _fromResponse(Response<dynamic>? response) {
    final code = response?.statusCode;
    final problem = ProblemDetails.tryParse(response?.data);

    if (code == 429) {
      return RateLimitedException(
        problem?.message ?? 'Too many requests. Please slow down.',
        retryAfter: parseRetryAfter(response?.headers.value('retry-after')),
      );
    }
    return ServerException(
      problem?.message ?? _defaultMessage(code),
      statusCode: code,
      fieldErrors: problem?.fieldErrors ?? const {},
    );
  }

  String _defaultMessage(int? code) => switch (code) {
    400 => 'Bad request.',
    401 => 'Please login again.',
    403 => 'You do not have access.',
    404 => 'Not found.',
    409 => 'Conflict: the data changed. Reload and try again.',
    422 => 'Validation failed.',
    final int c when c >= 500 => 'Server error. Please try later.',
    _ => 'Unexpected server response ($code).',
  };
}
```

`core/errors/failures.dart`: the same sealed family, now with `ValidationFailure.fieldErrors` and `RateLimitedFailure.retryAfter`

```
import 'package:equatable/equatable.dart';

import 'exceptions.dart';

/// What the domain / Cubit understands. [message] is always user-safe.
sealed class Failure extends Equatable implements Exception {
  const Failure(this.message, {this.cause, this.stackTrace});

  final String message;

  /// Technical reason. NEVER shown to users; forwarded to `Bloc.observer`.
  final Object? cause;
  final StackTrace? stackTrace;

  @override
```

```
  List<Object?> get props => [message];
}

class ServerFailure extends Failure {
  const ServerFailure(super.message, {this.statusCode});

  final int? statusCode;

  @override
  List<Object?> get props => [message, statusCode];
}

class UnauthorizedFailure extends Failure {
  const UnauthorizedFailure(super.message);
}

/// 422 / problem+json / client-side validation. [fieldErrors] maps a form
/// field to its messages so the UI can show them next to the input.
class ValidationFailure extends Failure {
  const ValidationFailure(super.message, {this.fieldErrors = const {}});

  final Map<String, List<String>> fieldErrors;

  @override
  List<Object?> get props => [message, fieldErrors];
}

class NetworkFailure extends Failure {
  const NetworkFailure(super.message);
}

/// 429: the UI can show "try again in N seconds".
class RateLimitedFailure extends Failure {
  const RateLimitedFailure(super.message, {this.retryAfter});

  final Duration? retryAfter;

  @override
  List<Object?> get props => [message, retryAfter];
}

class CancelledFailure extends Failure {
  const CancelledFailure() : super('Request cancelled.');
}

class ParsingFailure extends Failure {
  const ParsingFailure({super.cause, super.stackTrace})
    : super('Unexpected data received from server.');
}

class UnknownFailure extends Failure {
  const UnknownFailure({super.cause, super.stackTrace})
    : super('Something went wrong. Please try again.');
}

/// Repositories wrap every data-source call in this: Exception -> Failure.
Future<T> guard<T>(Future<T> Function() action) async {
  try {
    return await action();
  } on ServerException catch (e) {
    throw switch (e.statusCode) {
      401 || 403 => UnauthorizedFailure(e.message),
      422 => ValidationFailure(e.message, fieldErrors: e.fieldErrors),
      _ when e.fieldErrors.isNotEmpty => ValidationFailure(
        e.message,
        fieldErrors: e.fieldErrors,
      ),
      _ => ServerFailure(e.message, statusCode: e.statusCode),
    };
  } on RateLimitedException catch (e) {
    throw RateLimitedFailure(e.message, retryAfter: e.retryAfter);
  } on NetworkException catch (e) {
```

```
      throw NetworkFailure(e.message);
    } on CancelledException {
      throw const CancelledFailure();
    } on ParsingException catch (e, s) {
      throw ParsingFailure(cause: e, stackTrace: s);
    } on TypeError catch (e, s) {
      throw ParsingFailure(cause: e, stackTrace: s);
    } on FormatException catch (e, s) {
      throw ParsingFailure(cause: e, stackTrace: s);
    } catch (e, s) {
      throw UnknownFailure(cause: e, stackTrace: s);
    }
  }
```

`core/errors/problem_details.dart` : understands RFC 7807/9457 `problem+json` , `{"errors": {field: [..]}}` and `{"error": "..."}`

```
/// RFC 7807 / 9457 `application/problem+json`, plus the common
/// `{"errors": {"field": ["msg"]}}` validation shape.
class ProblemDetails {
  const ProblemDetails({
    this.type,
    this.title,
    this.status,
    this.detail,
    this.fieldErrors = const {},
  });

  final String? type;
  final String? title;
  final int? status;
  final String? detail;
  final Map<String, List<String>> fieldErrors;

  /// Best human-readable text the server gave us.
  String? get message => detail ?? title;

  /// Returns null when [body] is not a JSON object.
  static ProblemDetails? tryParse(Object? body) {
    if (body is! Map) return null;

    final fields = <String, List<String>>{};

    final errors = body['errors'];
    if (errors is Map) {
      errors.forEach((key, value) {
        fields['$key'] = value is List
            ? [for (final v in value) '$v']
            : ['$value'];
      });
    }

    final invalidParams = body['invalid-params']; // RFC 9457
    if (invalidParams is List) {
      for (final param in invalidParams) {
        if (param is Map && param['name'] != null) {
          (fields['${param['name']}'] ??= []).add('${param['reason']}');
        }
      }
    }

    String? text(String key) =>
        body[key] is String ? body[key] as String : null;

    return ProblemDetails(
      type: text('type'),
      title: text('title') ?? text('error'),
      status: body['status'] is int ? body['status'] as int : null,
      detail: text('detail') ?? text('message'),
      fieldErrors: fields,
```

```
    );
  }
}
```

LEVEL 1: EASY. A public list, no auth, no envelope

API: `GET https://jsonplaceholder.typicode.com/posts?_page=1&_limit=10` returns a **bare array** `[{...}, {...}]` (no `{"data": ...}`).

The 10-minute recipe

1. Paste one item into a comment. Note field types.
2. `Post` entity (plain Dart) + `PostModel` (`fromJson` + `toEntity`).
3. Repository interface (domain) → data source → repository impl (`guard`).
4. `GetPosts` use case → Cubit → page (loading / **empty** / error+Retry / list).

What is different about a bare list

- The model parses `(json as List)` ; there is no `data` key to unwrap.
- **Empty is a real state.** `[]` is a *successful* response. A blank screen is a bug; show "No posts found".
- Pagination metadata (if any) is **not in the body**: look in headers (Level 3).

`feature/posts/data/model/post_model.dart`

```dart
import '../../domain/entity/post.dart';

class PostModel {
  const PostModel({
    required this.id,
    required this.userId,
    required this.title,
    required this.body,
  });

  final int id;
  final int userId;
  final String title;
  final String body;

  /// This API returns a bare object / bare array: no `{"data": ...}` envelope.
  factory PostModel.fromJson(Map<String, dynamic> json) => PostModel(
    id: json['id'] as int,
    userId: json['userId'] as int,
    title: json['title'] as String,
    body: json['body'] as String,
  );

  Post toEntity() => Post(id: id, userId: userId, title: title, body: body);
}
```

`feature/posts/domain/usecase/get_posts.dart`

```dart
import '../../../../core/pagination/paged.dart';
import '../entity/post.dart';
import '../repository/post_repository.dart';

class GetPosts {
  const GetPosts(this._repository);

  final PostRepository _repository;

  Future<Paged<Post>> call({
    PageToken? after,
    int? userId,
    String query = '',
  }) => _repository.getPosts(after: after, userId: userId, query: query);
}
```

The state machine every list needs (written once, reused by every list)

`PaginatedState<T>` has **Initial / Loading / Empty / Error / Loaded** (+ `isLoadingMore`, `loadMoreFailed`). A feature never rewrites pagination; it only implements `fetchPage`:

`core/pagination/paginated_state.dart`

```dart
import 'package:equatable/equatable.dart';

import 'paged.dart';

sealed class PaginatedState<T> extends Equatable {
  const PaginatedState();

  @override
  List<Object?> get props => [];
}

class PaginatedInitial<T> extends PaginatedState<T> {
  const PaginatedInitial();
}

/// First page is loading (full-screen spinner).
class PaginatedLoading<T> extends PaginatedState<T> {
  const PaginatedLoading();
}

/// Loaded successfully but there is nothing to show: needs its own UI.
class PaginatedEmpty<T> extends PaginatedState<T> {
  const PaginatedEmpty();
}

class PaginatedError<T> extends PaginatedState<T> {
  const PaginatedError(this.message);

  final String message;

  @override
  List<Object?> get props => [message];
}

class PaginatedLoaded<T> extends PaginatedState<T> {
  const PaginatedLoaded({
    required this.items,
    this.next,
    this.isLoadingMore = false,
    this.loadMoreFailed = false,
  });

  final List<T> items;
  final PageToken? next;
  final bool isLoadingMore;
  final bool loadMoreFailed;

  bool get hasReachedMax => next == null;

  PaginatedLoaded<T> copyWith({
    List<T>? items,
    PageToken? next,
    bool clearNext = false,
    bool? isLoadingMore,
    bool? loadMoreFailed,
  }) => PaginatedLoaded<T>(
    items: items ?? this.items,
    next: clearNext ? null : (next ?? this.next),
    isLoadingMore: isLoadingMore ?? this.isLoadingMore,
    loadMoreFailed: loadMoreFailed ?? this.loadMoreFailed,
  );

  @override
```

```
  List<Object?> get props => [items, next, isLoadingMore, loadMoreFailed];
}
```

`feature/posts/presentation/cubit/posts_cubit.dart` : the whole list screen logic

```dart
import 'dart:math';

import '../../../../core/errors/failures.dart';
import '../../../../core/pagination/paged.dart';
import '../../../../core/pagination/paginated_cubit.dart';
import '../../../../core/pagination/paginated_state.dart';
import '../../domain/entity/post.dart';
import '../../domain/usecase/delete_post.dart';
import '../../domain/usecase/get_posts.dart';

/// The whole list screen: pagination is inherited, only `fetchPage` is ours.
class PostsCubit extends PaginatedCubit<Post> {
  PostsCubit(this._getPosts, this._deletePost);

  final GetPosts _getPosts;
  final DeletePost _deletePost;

  int? _userId;
  String _query = '';

  @override
  Future<Paged<Post>> fetchPage(PageToken? after) =>
      _getPosts(after: after, userId: _userId, query: _query);

  /// Server-side filter/search: reload from page 1.
  Future<void> applyFilter({int? userId, String? query}) {
    _userId = userId;
    _query = query?.trim() ?? '';
    return refresh();
  }

  /// OPTIMISTIC delete: the row disappears immediately; if the server says no,
  /// it comes back at the same position. Returns the failure (null = success)
  /// so the UI can show a snackbar.
  Future<Failure?> deletePost(Post post) async {
    final before = state;
    if (before is! PaginatedLoaded<Post>) return null;
    final index = before.items.indexOf(post);
    if (index < 0) return null;

    replaceItems([...before.items]..removeAt(index));
    try {
      await _deletePost(post.id);
      return null;
    } on Failure catch (failure) {
      reportFailure(failure);
      if (!isClosed) {
        final now = state;
        final items = switch (now) {
          PaginatedLoaded<Post>(:final items) => [...items],
          _ => <Post>[],
        };
        items.insert(min(index, items.length), post);
        replaceItems(items);
      }
      return failure;
    }
  }
}
```

Live proof (real API, through the whole stack)

```
L1 bare list      : 10 posts, total=100, next=_page=2&_limit=10
L3 follow Link    : page2 first id=11, next=_page=3&_limit=10
L3 last page      : reached page 10, hasNext=false, last id=100
```

```
L2 filter userId=3: 10 posts, all user 3: true, total=10
L2 search q       : 1 result(s), first="qui est esse"
L1 empty result   : items=0 (UI needs an Empty state)
L3 parallel       : post=1 author="Leanne Graham" comments=5
L2 POST           : id=101 title=Hello
L2 PUT            : id=1 title=Replaced
L2 PATCH          : title=Patched only body kept=true
L2 DELETE         : ok (no body to parse)
L2 404            : ServerFailure "Not found."
L4 ETag/304       : first fromCache=false, second fromCache=true, same data=true
```

(Lines `L1 …` are this level: a bare list of 10 with the total from a header, and a search with **no results** → an *empty list*, not an error.)

Level-1 pitfalls

Pitfall	Fix
`json['data']` on a bare list → `null` /crash	Cast the root: `(json as List)`
Empty list shows a blank screen	Separate **Empty** state with its own message
Wrong root type (server sent an object)	`TypeError` → `ParsingFailure` via `guard` (tested)
Trusting every field	Strict `as` casts: a broken contract should fail loudly

LEVEL 2: SIMPLE. Detail, filters, and the five verbs

2.1 Detail by id and query parameters

- `GET /posts/7` returns a **bare object** (still no envelope).
- Filters and search are query parameters: `?userId=3`, `?q=text`. Omit empty ones (`if (query.isNotEmpty) 'q': query`).
- The next-page URL (Level 3) already contains the filters, so changing a filter = **reload from page 1**.

2.2 The five verbs: what each one needs

Verb	Use	Body	Typical response	In code
`GET`	read	none	the resource / list	`client.get`
`POST`	create	request model	`201` + created resource (with new id)	`createPost`
`PUT`	**replace** whole resource	ALL fields	`200` + resource	`updatePost`
`PATCH`	change **some** fields	only changed fields	`200` + resource	`patchPost`, `{'title': ?title}`
`DELETE`	remove	none	`200 {}` or `204` (empty)	`deletePost` → `Future<void>`

`POST` is **not idempotent** (a retry can create a duplicate). The data source supports an `Idempotency-Key` **header** so a retried create can be recognised by the server.

`feature/posts/data/data_source/post_remote_data_source.dart`: every endpoint of the feature, 2–6 lines each

```dart
import '../../../../core/constants/api_constants.dart';
import '../../../../core/network/dio_client.dart';
import '../../../../core/network/link_header.dart';
import '../model/author_model.dart';
import '../model/comment_model.dart';
import '../model/post_model.dart';
import '../model/post_request_model.dart';
import '../model/posts_page_model.dart';

// Interface + impl in one file (LITE shortcut).
abstract interface class PostRemoteDataSource {
  Future<PostsPageModel> getPosts({String? nextUrl, int? userId, String query});
  Future<PostModel> getPost(int id);
  Future<PostModel> createPost(
    PostRequestModel request, {
    String? idempotencyKey,
  });
  Future<PostModel> updatePost(int id, PostRequestModel request);
  Future<PostModel> patchPost(int id, Map<String, dynamic> fields);
  Future<void> deletePost(int id);
  Future<List<CommentModel>> getComments(int postId);
  Future<AuthorModel> getAuthor(int userId);
}

class PostRemoteDataSourceImpl implements PostRemoteDataSource {
  const PostRemoteDataSourceImpl(this._client, {this.pageSize = 10});

  final DioClient _client;
  final int pageSize;

  @override
  Future<PostsPageModel> getPosts({
    String? nextUrl,
    int? userId,
    String query = '',
```

```
}) async {
  // Next page = follow the server's own link (filters are already inside it).
  final response = nextUrl != null
      ? await _client.getResponse(nextUrl)
      : await _client.getResponse(
          ApiConstants.posts,
          query: {
            '_page': 1,
            '_limit': pageSize,
            'userId': ?userId,
            if (query.isNotEmpty) 'q': query,
          },
        );

  return PostsPageModel(
    items: [
      for (final json in response.data as List)
        PostModel.fromJson(json as Map<String, dynamic>),
    ],
    nextUrl: parseLinkHeader(response.header('link'))['next'],
    total: int.tryParse(response.header('x-total-count') ?? ''),
  );
}

@override
Future<PostModel> getPost(int id) async => PostModel.fromJson(
  await _client.get('${ApiConstants.posts}/$id') as Map<String, dynamic>,
);

@override
Future<PostModel> createPost(
  PostRequestModel request, {
  String? idempotencyKey,
}) async => PostModel.fromJson(
  await _client.post(
    ApiConstants.posts,
    data: request.toJson(),
    headers: {'Idempotency-Key': ?idempotencyKey},
  ) as Map<String, dynamic>,
);

@override
Future<PostModel> updatePost(int id, PostRequestModel request) async =>
    PostModel.fromJson(
      await _client.put('${ApiConstants.posts}/$id', data: request.toJson())
          as Map<String, dynamic>,
    );

@override
Future<PostModel> patchPost(int id, Map<String, dynamic> fields) async =>
    PostModel.fromJson(
      await _client.patch('${ApiConstants.posts}/$id', data: fields)
          as Map<String, dynamic>,
    );

@override
Future<void> deletePost(int id) async {
  await _client.delete(
    '${ApiConstants.posts}/$id',
  ); // body is `{}`: ignore it
}

@override
Future<List<CommentModel>> getComments(int postId) async => [
  for (final json
      in await _client.get('${ApiConstants.posts}/$postId/comments') as List)
    CommentModel.fromJson(json as Map<String, dynamic>),
];

@override
Future<AuthorModel> getAuthor(int userId) async => AuthorModel.fromJson(
  await _client.get('${ApiConstants.users}/$userId') as Map<String, dynamic>,
```

```
  );
}
```

2.3 A form is a COMMAND, not a query

A *query* screen **is** the data (Loading/Loaded/Error). A *command* screen is a form whose result is a **one-time event**. States: `Idle → Submitting → Saved | Invalid(fieldErrors) | Failed`. Use `BlocConsumer`: **listener** for one-offs (pop, snackbar), **builder** for the form.

2.4 Validation: one shape for client AND server errors

The domain validates first (`PostDraft.validate()`), returning `field → messages`. A server `422` problem+json is mapped to the **same** `ValidationFailure.fieldErrors`. So the form shows errors under the right field with **one** code path.

`feature/posts/domain/entity/post_draft.dart`

```dart
import 'package:equatable/equatable.dart';

/// What the user typed. Business rules live here, and they report errors in
/// the SAME shape as server-side validation (field -> messages).
class PostDraft extends Equatable {
  const PostDraft({required this.title, required this.body, this.userId = 1});

  final String title;
  final String body;
  final int userId;

  Map<String, List<String>> validate() {
    final errors = <String, List<String>>{};
    final t = title.trim();
    if (t.isEmpty) {
      errors['title'] = ['Title is required'];
    } else if (t.length > 100) {
      errors['title'] = ['Title must be 100 characters or less'];
    }
    if (body.trim().isEmpty) errors['body'] = ['Body is required'];
    return errors;
  }

  @override
  List<Object?> get props => [title, body, userId];
}
```

`feature/posts/domain/usecase/save_post.dart`: create or update, rules checked **before** any request

```dart
import '../../../../core/errors/failures.dart';
import '../entity/post.dart';
import '../entity/post_draft.dart';
import '../repository/post_repository.dart';

/// Create or update, with the business rules checked BEFORE any request.
/// Client-side and server-side validation arrive as the same
/// `ValidationFailure.fieldErrors`, so the form code is written once.
class SavePost {
  const SavePost(this._repository);

  final PostRepository _repository;

  Future<Post> call(PostDraft draft, {int? editingId, String? idempotencyKey}) {
    final errors = draft.validate();
    if (errors.isNotEmpty) {
      throw ValidationFailure(
        'Please fix the highlighted fields.',
        fieldErrors: errors,
      );
    }
    return editingId == null
```

```
        ? _repository.createPost(draft, idempotencyKey: idempotencyKey)
        : _repository.updatePost(editingId, draft);
  }
}
```

`feature/posts/presentation/cubit/post_editor_cubit.dart`

```dart
import 'package:equatable/equatable.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../../../../core/errors/failures.dart';
import '../../../../core/errors/report_failure.dart';
import '../../domain/entity/post.dart';
import '../../domain/entity/post_draft.dart';
import '../../domain/usecase/save_post.dart';

/// A COMMAND screen (a form), not a query: Idle -> Submitting -> Saved/...
sealed class PostEditorState extends Equatable {
  const PostEditorState();

  @override
  List<Object?> get props => [];
}

class EditorIdle extends PostEditorState {
  const EditorIdle();
}

class EditorSubmitting extends PostEditorState {
  const EditorSubmitting();
}

class EditorSaved extends PostEditorState {
  const EditorSaved(this.post);

  final Post post;

  @override
  List<Object?> get props => [post];
}

/// Client-side OR server-side validation: field -> messages, shown inline.
class EditorInvalid extends PostEditorState {
  const EditorInvalid(this.message, this.fieldErrors);

  final String message;
  final Map<String, List<String>> fieldErrors;

  @override
  List<Object?> get props => [message, fieldErrors];
}

class EditorFailed extends PostEditorState {
  const EditorFailed(this.message);

  final String message;

  @override
  List<Object?> get props => [message];
}

class PostEditorCubit extends Cubit<PostEditorState>
    with FailureReporting<PostEditorState> {
  PostEditorCubit(this._savePost, {this.editing}) : super(const EditorIdle());

  final SavePost _savePost;

  /// null = creating a new post, otherwise editing (PUT) this one.
  final Post? editing;

  Future<void> submit(PostDraft draft) async {
```

```
    if (state is EditorSubmitting) return; // ignore double taps
    emit(const EditorSubmitting());
    try {
      final saved = await _savePost(draft, editingId: editing?.id);
      if (!isClosed) emit(EditorSaved(saved));
    } on ValidationFailure catch (failure) {
      if (!isClosed) emit(EditorInvalid(failure.message, failure.fieldErrors));
    } on Failure catch (failure) {
      reportFailure(failure);
      if (!isClosed) emit(EditorFailed(failure.message));
    }
  }
}
```

2.5 Optimistic delete with rollback

The row disappears **immediately**; if the server refuses, it comes back at the **same position** and the UI shows a snackbar. (This is `PostsCubit.deletePost`, shown in Level 1 and tested for: success, failure restores, deleting the only item shows Empty then restores, unknown post is ignored.)

Live proof

```
L1 bare list      : 10 posts, total=100, next=_page=2&_limit=10
L3 follow Link    : page2 first id=11, next=_page=3&_limit=10
L3 last page      : reached page 10, hasNext=false, last id=100
L2 filter userId=3: 10 posts, all user 3: true, total=10
L2 search q       : 1 result(s), first="qui est esse"
L1 empty result   : items=0 (UI needs an Empty state)
L3 parallel       : post=1 author="Leanne Graham" comments=5
L2 POST           : id=101 title=Hello
L2 PUT            : id=1 title=Replaced
L2 PATCH          : title=Patched only body kept=true
L2 DELETE         : ok (no body to parse)
L2 404            : ServerFailure "Not found."
L4 ETag/304       : first fromCache=false, second fromCache=true, same data=true
```

(Lines `L2 …`: filter `userId=3` returns only user 3's posts; search `q`; `POST` → id 101; `PUT`; `PATCH` changes only the title and keeps the body; `DELETE` with no body to parse; `404` → `ServerFailure "Not found."`)

Level-2 pitfalls

Pitfall	Fix
`PATCH` sends every field	Send only provided fields: `{'title': ?title}` (null-aware map element)
Parsing the body of a `204` / `{}` delete	Ignore it: `await _client.delete(...)`
Double-tap on Save creates two posts	Cubit ignores `submit` while `Submitting` (tested); send an `Idempotency-Key`
Server field errors shown as one generic toast	Map to `fieldErrors`, show under inputs
Optimistic UI that never rolls back	Always restore on `Failure` and tell the user

---

LEVEL 3: MEDIUM. Pagination, nested resources, files

3.1 Three kinds of pagination: one abstraction

The API dictates how to ask for the next page. The app asks it the same way every time:

Style	Request	Where the "next" info is	`PageToken`
Page number	`?page=3`	`total_pages` in the body	`PageNumber(3)`
Cursor	`?after=<opaque>`	`next_cursor` in the body	`CursorToken('abc')`
Next URL	follow the URL	`Link: <…>; rel="next"` **header**	`NextUrl('https://…')`

`core/pagination/paged.dart`

```dart
import 'package:equatable/equatable.dart';

/// "How to ask for the next page". Which one you use is dictated by the API:
///   page number  -> `?page=3`           (reqres, most CRUD APIs)
///   cursor       -> `?after=<token>`    (Stripe, Slack, Graph APIs)
///   next URL     -> follow `Link: <...>; rel="next"` (GitHub, JSONPlaceholder)
sealed class PageToken extends Equatable {
  const PageToken();
}

class PageNumber extends PageToken {
  const PageNumber(this.page);

  final int page;

  @override
  List<Object?> get props => [page];
}

class CursorToken extends PageToken {
  const CursorToken(this.cursor);

  final String cursor;

  @override
  List<Object?> get props => [cursor];
}

class NextUrl extends PageToken {
  const NextUrl(this.url);

  final String url;

  @override
  List<Object?> get props => [url];
}

/// One page of any list + the token for the next one (null = last page).
class Paged<T> extends Equatable {
  const Paged({required this.items, this.next, this.total});

  final List<T> items;
  final PageToken? next;
  final int? total;

  bool get hasNext => next != null;

  @override
  List<Object?> get props => [items, next, total];
}
```

The `Link` header is parsed by a tiny pure function (RFC 8288), tested against the **real** header from the API:

`core/network/link_header.dart`

```dart
/// RFC 8288 `Link` header -> {rel: url}.
/// `<https://api/x?page=3>; rel="next", <https://api/x?page=9>; rel="last"`
Map<String, String> parseLinkHeader(String? header) {
  final links = <String, String>{};
  if (header == null || header.trim().isEmpty) return links;

  // Split on commas that start a new `<url>` (URLs may contain commas).
  for (final part in header.split(RegExp(r',\s*(?=<)'))) {
    final url = RegExp(r'<([^>]*)>').firstMatch(part)?.group(1);
    final rel = RegExp(r'rel="?([^";]+)"?').firstMatch(part)?.group(1);
    if (url != null && rel != null) {
      for (final r in rel.split(' ')) {
        links[r] = url;
      }
    }
  }
  return links;
}
```

`PaginatedCubit` is **generic**: it works unchanged with all three token types (verified by a test that runs the same Cubit with each one), and it already handles: empty first page, page-2 failure (list kept, retry), duplicate `loadMore` calls, and a reload while a page is in flight.

`core/pagination/paginated_cubit.dart`

```dart
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:meta/meta.dart';

import '../errors/failures.dart';
import '../errors/report_failure.dart';
import 'paged.dart';
import 'paginated_state.dart';

/// Pagination written ONCE for every list endpoint. A feature only implements
/// [fetchPage]: all page-number / cursor / next-url APIs work the same way.
abstract class PaginatedCubit<T> extends Cubit<PaginatedState<T>>
    with FailureReporting<PaginatedState<T>> {
  PaginatedCubit() : super(PaginatedInitial<T>());

  /// `after == null` means "first page".
  @protected
  Future<Paged<T>> fetchPage(PageToken? after);

  int _generation = 0; // a newer refresh() invalidates older responses

  /// First page (initial load, Retry, pull-to-refresh, filter change).
  Future<void> refresh() async {
    final generation = ++_generation;
    emit(PaginatedLoading<T>());
    try {
      final page = await fetchPage(null);
      if (isClosed || generation != _generation) return;
      emit(
        page.items.isEmpty
            ? PaginatedEmpty<T>()
            : PaginatedLoaded<T>(items: page.items, next: page.next),
      );
    } on CancelledFailure {
      return;
    } on Failure catch (failure) {
      reportFailure(failure);
      if (isClosed || generation != _generation) return;
      emit(PaginatedError<T>(failure.message));
    }
  }

  /// Next page, appended. Safe to call as often as the UI likes.
  Future<void> loadMore() async {
```

```
    final current = state;
    if (current is! PaginatedLoaded<T> ||
        current.isLoadingMore ||
        current.hasReachedMax) {
      return;
    }

    final generation = _generation;
    emit(current.copyWith(isLoadingMore: true, loadMoreFailed: false));
    try {
      final page = await fetchPage(current.next);
      if (isClosed || generation != _generation) return;
      emit(
        PaginatedLoaded<T>(
          items: [...current.items, ...page.items],
          next: page.next,
        ),
      );
    } on CancelledFailure {
      return;
    } on Failure catch (failure) {
      reportFailure(failure);
      if (isClosed || generation != _generation) return;
      emit(current.copyWith(isLoadingMore: false, loadMoreFailed: true));
    }
  }

  /// For optimistic edits (delete/insert/undo) on the list.
  @protected
  void replaceItems(List<T> items) {
    final current = state;
    switch (current) {
      case PaginatedLoaded<T>():
        emit(
          items.isEmpty && current.hasReachedMax
              ? PaginatedEmpty<T>()
              : current.copyWith(items: items),
        );
      case PaginatedEmpty<T>():
        if (items.isNotEmpty) emit(PaginatedLoaded<T>(items: items));
      default:
        return;
    }
  }
}
```

3.2 Pagination info in headers: `getResponse`

A bare list cannot carry `total` or `next`, so the server puts them in headers. `DioClient.getResponse` returns **body + headers**; the data source reads `x-total-count` and `Link`:

`feature/posts/data/model/posts_page_model.dart`

```
import 'post_model.dart';

/// Page of a BARE LIST: the pagination info lives in HEADERS, not the body.
class PostsPageModel {
  const PostsPageModel({required this.items, this.nextUrl, this.total});

  final List<PostModel> items;
  final String? nextUrl; // from `Link: <...>; rel="next"`
  final int? total; // from `x-total-count`
}
```

(`PostRemoteDataSource.getPosts` in Level 2 shows the call: first page with `_page` / `_limit`, later pages by **following the server's own URL verbatim**: filters are already inside it.)

3.3 Several calls for one screen

A post-details screen needs the post, its author, and its comments:

Call	Depends on	Strategy
comments	only the post id	start **now**, in parallel
post	nothing	**required**: its failure fails the screen
author	`post.userId`	**dependent**: starts after the post

Author and comments are **optional sections**: if one fails, the screen still renders (`null` = "couldn't load"). One Dart 3 record waits for both: `final (a, c) = await (authorFuture, commentsFuture).wait;`.

`feature/posts/domain/usecase/get_post_details.dart`

```dart
import '../../../../core/errors/failures.dart';
import '../entity/post_details.dart';
import '../repository/post_extras_repository.dart';
import '../repository/post_repository.dart';

/// Several calls, one screen:
///   comments  -> independent of the post   -> start NOW, in parallel
///   post      -> REQUIRED                  -> failure fails the whole screen
///   author    -> needs `post.userId`       -> dependent: starts after the post
/// Author and comments are OPTIONAL: if one fails the screen still renders.
class GetPostDetails {
  const GetPostDetails(this._posts, this._extras);

  final PostRepository _posts;
  final PostExtrasRepository _extras;

  Future<PostDetails> call(int id) async {
    final comments = _optional(() => _extras.getComments(id)); // independent
    final post = await _posts.getPost(id); // required
    final author = _optional(() => _extras.getAuthor(post.userId)); // dependent

    final (loadedAuthor, loadedComments) = await (author, comments).wait;
    return PostDetails(
      post: post,
      author: loadedAuthor,
      comments: loadedComments,
    );
  }

  Future<T?> _optional<T>(Future<T> Function() load) async {
    try {
      return await load();
    } on Failure {
      return null; // degrade gracefully
    }
  }
}
```

Tested: all succeed; **comments are requested before the post finishes**; the author is requested with the post's `userId`; comments fail → screen still loads; author fails → rest fine; the required post fails → whole call fails.

3.4 Files: upload with progress and cancel; download

- Upload = `FormData` + `MultipartFile`, with `onSendProgress`. Progress is **throttled** to whole percents (otherwise you rebuild the UI hundreds of times).
- The domain sees an `UploadTask` (`progress`, `result`, `cancel()`), with no `Dio`, `CancelToken` or `FormData` leaking out.
- Leaving the screen **cancels** the upload (`close()`).

- A bug found by the tests: awaiting `StreamController.close()` never completes when nobody listens to progress, which would hang `result`. It is fixed (and covered).

`feature/files/data/data_source/file_remote_data_source.dart`

```dart
import 'dart:async';
import 'dart:typed_data';

import 'package:dio/dio.dart';

import '../../../../core/constants/api_constants.dart';
import '../../../../core/network/dio_client.dart';
import '../model/upload_response_model.dart';

/// What the data layer returns for a running upload (still model-level).
class RemoteUpload {
  const RemoteUpload({
    required this.progress,
    required this.result,
    required this.cancel,
  });

  final Stream<double> progress;
  final Future<UploadResponseModel> result;
  final void Function() cancel;
}

class FileRemoteDataSource {
  const FileRemoteDataSource(this._client);

  final DioClient _client;

  RemoteUpload upload(Uint8List bytes, String fileName) {
    final token = CancelToken();
    final progress = StreamController<double>();
    var lastPercent = -1;

    final result = () async {
      try {
        final form = FormData.fromMap({
          'file': MultipartFile.fromBytes(bytes, filename: fileName),
        });
        final json = await _client.post(
          ApiConstants.uploadUrl,
          data: form,
          cancelToken: token,
          onSendProgress: (sent, total) {
            if (total <= 0) return;
            final percent = (sent * 100 ~/ total); // throttle: whole percents
            if (percent != lastPercent && !progress.isClosed) {
              lastPercent = percent;
              progress.add(sent / total);
            }
          },
        );
        return UploadResponseModel.fromJson(json as Map<String, dynamic>);
      } finally {
        // Not awaited: close() only completes once a listener has consumed
        // the done event, which would hang `result` for callers who ignore
        // progress.
        unawaited(progress.close());
      }
    }();

    return RemoteUpload(
      progress: progress.stream,
      result: result,
      cancel: () => token.cancel('user cancelled'),
    );
  }
```

```
  /// Download to a file, reporting 0..1 progress when the size is known.
  Future<void> download(
    String url,
    String savePath, {
    void Function(double progress)? onProgress,
    CancelToken? cancelToken,
  }) => _client.download(
    url,
    savePath,
    cancelToken: cancelToken,
    onReceiveProgress: (received, total) {
      if (total > 0) onProgress?.call(received / total);
    },
  );
}
```

`feature/files/presentation/cubit/upload_cubit.dart`

```
import 'dart:typed_data';

import 'package:equatable/equatable.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../../../../core/errors/failures.dart';
import '../../../../core/errors/report_failure.dart';
import '../../domain/entity/upload_receipt.dart';
import '../../domain/entity/upload_task.dart';
import '../../domain/usecase/upload_file.dart';

sealed class UploadState extends Equatable {
  const UploadState();

  @override
  List<Object?> get props => [];
}

class UploadIdle extends UploadState {
  const UploadIdle();
}

class UploadInProgress extends UploadState {
  const UploadInProgress(this.progress);

  final double progress; // 0..1

  @override
  List<Object?> get props => [progress];
}

class UploadDone extends UploadState {
  const UploadDone(this.receipt);

  final UploadReceipt receipt;

  @override
  List<Object?> get props => [receipt];
}

class UploadFailed extends UploadState {
  const UploadFailed(this.message);

  final String message;

  @override
  List<Object?> get props => [message];
}

class UploadCubit extends Cubit<UploadState>
    with FailureReporting<UploadState> {
  UploadCubit(this._uploadFile) : super(const UploadIdle());
```

```
  final UploadFile _uploadFile;
  UploadTask? _task;

  Future<void> start(Uint8List bytes, String fileName) async {
    if (state is UploadInProgress) return; // one upload at a time
    emit(const UploadInProgress(0));

    final task = _task = _uploadFile(bytes: bytes, fileName: fileName);
    final subscription = task.progress.listen((value) {
      // Only while uploading: a late event must never overwrite Done/Failed.
      if (!isClosed && state is UploadInProgress) emit(UploadInProgress(value));
    });

    try {
      final receipt = await task.result;
      if (!isClosed) emit(UploadDone(receipt));
    } on CancelledFailure {
      if (!isClosed) emit(const UploadIdle());
    } on Failure catch (failure) {
      reportFailure(failure);
      if (!isClosed) emit(UploadFailed(failure.message));
    } finally {
      await subscription.cancel();
      _task = null;
    }
  }

  void cancel() => _task?.cancel();

  @override
  Future<void> close() {
    _task?.cancel(); // leaving the screen aborts the upload
    return super.close();
  }
}
```

Live proof (real servers)

```
upload   : sample.bin 300000 bytes, 3 progress events, last=1.0, monotonic=true
cancel   : CancelledFailure (request aborted)
download : 20000 bytes on disk, progress events=4, last=1.0
```

and, for pagination, the `L3 …` lines of the Level-1 proof run: following `Link` from page 1 to page **10** (`hasNext=false` on the last page).

Level-3 pitfalls

Pitfall	Fix
Re-implementing `loadMore` per screen	One `PaginatedCubit` ; the feature supplies `fetchPage`
Building the next URL by hand	Follow `Link rel="next"` verbatim
Scroll listener never fires on a short first page	Footer widget that triggers when **built**
Page-2 error wipes the list	Keep items, show Retry in the footer
Progress events flood the UI	Throttle to whole percents
Sequential calls that could be parallel	Independent → start together; dependent → after

LEVEL 4: HARD. Shapes, standards, throttling, caching, GraphQL

4.1 Polymorphic JSON: one list, several shapes

`{"items": [{"type": "text"...}, {"type": "image"...}, {"type": "poll"...}]}`

- A **sealed** domain class (`TextItem` , `ImageItem` , `PollItem`) → the UI `switch` must cover every kind (compile-time).
- An unknown `type` becomes `UnknownItem` (forward compatible: the app doesn't crash when the server ships a new type; the UI says "Update the app").
- **Tolerant list parsing:** one malformed item is **skipped and counted**; the rest still render.

`feature/feed/data/model/feed_item_model.dart`

```dart
import '../../domain/entity/feed_item.dart';

/// Decides the Dart type from the JSON `type` discriminator.
abstract final class FeedItemModel {
  static FeedItem fromJson(Map<String, dynamic> json) {
    final id = '${json['id']}';
    final type = json['type'];
    return switch (type) {
      'text' => TextItem(id, json['text'] as String),
      'image' => ImageItem(
        id,
        json['url'] as String,
        width: (json['width'] as num?)?.toInt(),
        height: (json['height'] as num?)?.toInt(),
      ),
      'poll' => PollItem(id, json['question'] as String, [
        for (final o in json['options'] as List)
          PollOption(
            (o as Map<String, dynamic>)['label'] as String,
            o['votes'] as int? ?? 0,
          ),
      ]),
      // New server-side type (e.g. "video"): keep it, don't crash.
      final String other => UnknownItem(id, other),
      _ => throw const FormatException('feed item without a "type"'),
    };
  }
}
```

`feature/feed/data/model/feed_response_model.dart`

```dart
import '../../domain/entity/feed_item.dart';
import 'feed_item_model.dart';

/// Tolerant list parsing: ONE broken item must not blank the whole feed.
class FeedResponseModel {
  const FeedResponseModel({required this.items, required this.skipped});

  final List<FeedItem> items;
  final int skipped;

  factory FeedResponseModel.fromJson(Map<String, dynamic> json) {
    final items = <FeedItem>[];
    var skipped = 0;
    for (final raw in json['items'] as List) {
      try {
        items.add(FeedItemModel.fromJson(raw as Map<String, dynamic>));
      } on TypeError {
        skipped++; // wrong field type / missing required field
      } on FormatException {
        skipped++; // no discriminator
      }
    }
    return FeedResponseModel(items: items, skipped: skipped);
```

```
  }

  FeedPage toEntity() => FeedPage(items: items, skipped: skipped);
}
```

4.2 Standard error bodies (RFC 7807 / 9457 problem+json)

`ProblemDetails` (shown in A.3) reads `title` / `detail`, `errors: {field: [...]}` and `invalid-params`. `DioClient` turns them into `ServerException(message, fieldErrors)`, `guard` into `ValidationFailure`. OAuth-style `{"error": "...", "error_description": "..."}` and plain `{"message": ...}` bodies work as well.

4.3 `429 Too Many Requests` and `Retry-After`

`Retry-After` is **seconds or an HTTP date**. The interceptor waits (capped by `maxWait`), retries via a plain Dio (no loops), gives up after `maxRetries`, and then `DioClient` raises `RateLimitedException(retryAfter)` so the UI can say "try again in N s".

`core/network/retry_after.dart` and `retry_after_interceptor.dart`

```dart
import 'dart:io' show HttpDate, HttpException;

/// `Retry-After` is either delta-seconds ("120") or an HTTP date.
Duration? parseRetryAfter(String? value, {DateTime Function()? now}) {
  if (value == null) return null;
  final trimmed = value.trim();
  final seconds = int.tryParse(trimmed);
  if (seconds != null) return Duration(seconds: seconds < 0 ? 0 : seconds);
  try {
    final wait = HttpDate.parse(trimmed).difference((now ?? DateTime.now)());
    return wait.isNegative ? Duration.zero : wait;
  } on HttpException {
    return null;
  } on FormatException {
    return null;
  }
}
```

```dart
import 'package:dio/dio.dart';

import 'retry_after.dart';

/// 429 Too Many Requests: wait for `Retry-After` (capped), then retry.
/// Gives up after [maxRetries] and lets the 429 through to DioClient, which
/// turns it into a RateLimitedException.
class RetryAfterInterceptor extends Interceptor {
  RetryAfterInterceptor({
    required this.retryDio,
    this.maxRetries = 2,
    this.maxWait = const Duration(seconds: 30),
    Future<void> Function(Duration)? delay,
  }) : _delay = delay ?? Future<void>.delayed;

  /// A Dio WITHOUT this interceptor (no retry loops).
  final Dio retryDio;
  final int maxRetries;
  final Duration maxWait;
  final Future<void> Function(Duration) _delay;

  @override
  Future<void> onError(
    DioException err,
    ErrorInterceptorHandler handler,
  ) async {
    if (err.response?.statusCode != 429) return handler.next(err);

    var current = err;
    for (var attempt = 1; attempt <= maxRetries; attempt++) {
      final suggested =
```

```
          parseRetryAfter(current.response?.headers.value('retry-after')) ??
          Duration(seconds: 1 << (attempt - 1)); // 1s, 2s, 4s...
      await _delay(suggested > maxWait ? maxWait : suggested);
      try {
        return handler.resolve(
          await retryDio.fetch<dynamic>(err.requestOptions),
        );
      } on DioException catch (e) {
        if (e.response?.statusCode != 429) return handler.next(e);
        current = e;
      }
    }
    handler.next(current);
  }
}
```

Tested with an injected fake clock/delay (no real waiting): honours the header, exponential 1 s/2 s when absent, caps huge values, gives up after N, never retries non-429.

4.4 HTTP caching with ETag (conditional GET)

First response carries `ETag`. Next request sends `If-None-Match`; if the server replies `304 Not Modified`, the interceptor returns the **cached body**: no re-download, and the caller never sees the 304 (`ApiResponse.fromCache == true`).

`core/network/etag_cache_interceptor.dart`

```
import 'package:dio/dio.dart';

/// HTTP conditional GET: remember `ETag` + body per URL, send `If-None-Match`,
/// and turn a `304 Not Modified` back into the cached body (no re-download).
class ETagCacheInterceptor extends Interceptor {
  final _cache = <String, ({String etag, dynamic data})>{};

  String _key(RequestOptions o) => o.uri.toString();

  void clear() => _cache.clear();

  @override
  void onRequest(RequestOptions options, RequestInterceptorHandler handler) {
    final hit = options.method == 'GET' ? _cache[_key(options)] : null;
    if (hit != null) options.headers['If-None-Match'] = hit.etag;
    handler.next(options);
  }

  @override
  void onResponse(
    Response<dynamic> response,
    ResponseInterceptorHandler handler,
  ) {
    final etag = response.headers.value('etag');
    if (response.requestOptions.method == 'GET' &&
        response.statusCode == 200 &&
        etag != null) {
      _cache[_key(response.requestOptions)] = (etag: etag, data: response.data);
    }
    handler.next(response);
  }

  @override
  void onError(DioException err, ErrorInterceptorHandler handler) {
    final response = err.response;
    final hit = _cache[_key(err.requestOptions)];
    if (response?.statusCode == 304 && hit != null) {
      return handler.resolve(
        Response<dynamic>(
          requestOptions: err.requestOptions,
          data: hit.data,
          statusCode: 200,
```

```
          headers: response!.headers,
          extra: {'fromCache': true},
        ),
      );
    }
    handler.next(err);
  }
}
```

4.5 Stale-while-revalidate: instant screen, then fresh data

Show cached data **immediately** (marked stale), fetch, then replace. If the refresh fails with a cache present, keep showing stale data **plus** the failure. A broken cache (read/write error) must **never** break loading.

`core/cache/stale_while_revalidate.dart`

```dart
import 'package:equatable/equatable.dart';

import '../errors/failures.dart';

/// A value plus "how fresh is it".
class Cached<T> extends Equatable {
  const Cached(this.value, {required this.isStale, this.refreshFailure});

  final T value;

  /// true = from the cache, a network refresh is pending or has failed.
  final bool isStale;

  /// Set when we are showing cached data BECAUSE the refresh failed.
  final Failure? refreshFailure;

  @override
  List<Object?> get props => [value, isStale, refreshFailure];
}

/// Show cached data instantly, then replace it with fresh data.
///
///   cache hit    -> emit(stale) ... fetch ... emit(fresh)
///   cache miss   -> fetch ... emit(fresh)
///   fetch fails  -> with a cache: emit(stale + refreshFailure), else rethrow
///
/// A broken cache (read/write error) must NEVER break loading.
Stream<Cached<T>> staleWhileRevalidate<T>({
  required Future<T?> Function() readCache,
  required Future<T> Function() fetch,
  required Future<void> Function(T value) writeCache,
}) async* {
  T? cached;
  try {
    cached = await readCache();
  } catch (_) {
    cached = null;
  }
  if (cached != null) yield Cached(cached, isStale: true);

  try {
    final fresh = await fetch();
    try {
      await writeCache(fresh);
    } catch (_) {
      // ignore: the fresh data is still good for this session
    }
    yield Cached(fresh, isStale: false);
  } on Failure catch (failure) {
    if (cached == null) rethrow;
    yield Cached(cached, isStale: true, refreshFailure: failure);
  }
}
```

`feature/posts/domain/usecase/watch_post.dart`

```dart
import '../../../../core/cache/stale_while_revalidate.dart';
import '../entity/post.dart';
import '../repository/post_cache.dart';
import '../repository/post_repository.dart';

/// Cached post instantly (marked stale), then the fresh one.
class WatchPost {
  const WatchPost(this._repository, this._cache);

  final PostRepository _repository;
  final PostCache _cache;

  Stream<Cached<Post>> call(int id) => staleWhileRevalidate<Post>(
    readCache: () => _cache.read(id),
    fetch: () => _repository.getPost(id),
    writeCache: _cache.write,
  );
}
```

4.6 GraphQL: HTTP 200 does NOT mean success

GraphQL is one `POST` to one URL with `{query, variables, operationName}`. The trap: **errors come back with HTTP 200** in an `errors` array. The client checks the body, translates `extensions.code` (`UNAUTHENTICATED` →401, `FORBIDDEN` →403, `BAD_USER_INPUT` →422, else 400) so the rest of the app (Failures, 401 handling) works unchanged, and lets you opt into **partial data**. Also: an unknown id returns `data.country == null`, which is **not** an error.

`core/network/graphql_client.dart`

```dart
import '../errors/exceptions.dart';
import 'dio_client.dart';

/// GraphQL over HTTP. The trap: the server answers **HTTP 200 even when the
/// operation failed**; the failure is in the body's `errors` array.
class GraphQLClient {
  const GraphQLClient(this._client, this.endpoint);

  final DioClient _client;
  final String endpoint;

  /// Returns the `data` object.
  /// [allowPartialData]: GraphQL may return data AND errors (one field failed).
  Future<Map<String, dynamic>> query(
    String document, {
    Map<String, dynamic>? variables,
    String? operationName,
    bool allowPartialData = false,
  }) async {
    final json = await _client.post(
      endpoint,
      data: {
        'query': document,
        'variables': ?variables,
        'operationName': ?operationName,
      },
    );
    if (json is! Map<String, dynamic>) {
      throw const ParsingException('GraphQL response is not an object.');
    }

    final errors = json['errors'];
    final data = json['data'];
    final failed = errors is List && errors.isNotEmpty;
    if (failed && (data == null || !allowPartialData)) {
      throw _toException(errors);
    }
    if (data is! Map<String, dynamic>) {
      throw const ParsingException('GraphQL response has no data.');
```

```
    }
    return data;
  }

  /// GraphQL has no status codes: translate `extensions.code` into ours so the
  /// rest of the app (Failures, 401 handling...) works unchanged.
  ServerException _toException(List<dynamic> errors) {
    final first = errors.first;
    final error = first is Map<String, dynamic>
        ? first
        : const <String, dynamic>{};
    final message = error['message'] is String
        ? error['message'] as String
        : 'GraphQL request failed.';
    final extensions = error['extensions'];
    final code = extensions is Map ? extensions['code'] : null;

    return ServerException(
      message,
      statusCode: switch (code) {
        'UNAUTHENTICATED' => 401,
        'FORBIDDEN' => 403,
        'BAD_USER_INPUT' => 422,
        _ => 400,
      },
    );
  }
}
```

`feature/countries/data/data_source/country_remote_data_source.dart` : the queries live next to the data source

```
import '../../../../core/network/graphql_client.dart';
import '../model/country_model.dart';

/// GraphQL documents live next to the data source that owns them.
const _countryQuery = r'''
query Country($code: ID!) {
  country(code: $code) { code name capital emoji }
}''';

const _byContinentQuery = r'''
query ByContinent($continent: String!) {
  countries(filter: { continent: { eq: $continent } }) { code name capital emoji }
}''';

class CountryRemoteDataSource {
  const CountryRemoteDataSource(this._graphql);

  final GraphQLClient _graphql;

  Future<CountryModel?> getCountry(String code) async {
    final data = await _graphql.query(
      _countryQuery,
      variables: {'code': code},
      operationName: 'Country',
    );
    final country = data['country'];
    return country == null
        ? null
        : CountryModel.fromJson(country as Map<String, dynamic>);
  }

  Future<List<CountryModel>> getByContinent(String continentCode) async {
    final data = await _graphql.query(
      _byContinentQuery,
      variables: {'continent': continentCode},
      operationName: 'ByContinent',
    );
    return [
      for (final json in data['countries'] as List)
        CountryModel.fromJson(json as Map<String, dynamic>),
```

```
    ];
  }
}
```

Live proof (real servers)

```
query ok      : India / New Delhi 🇮🇳
unknown code  : null  (data.country == null is NOT an error)
list + filter : 52 countries in Asia, first=United Arab Emirates
invalid query : HTTP 200 but threw ServerException: Cannot query field "nope" on type "Query". (status 400)
```

and the `L4 ETag/304` line of the Level-1 proof run (`second fromCache=true`).

Level-4 pitfalls

Pitfall	Fix
`switch` on JSON `type` with a `default` that hides new types	Keep unknown types (`UnknownItem`), render a graceful placeholder
One bad item blanks the whole feed	Parse per item; skip + count
Treating GraphQL HTTP 200 as success	Always inspect `errors`
Retrying 4xx forever	Retry only network/5xx/429
Caching in widgets	Cache in the repository/use case (SWR), UI just shows `isStale`
`Retry-After` assumed to be seconds	Parse seconds **and** HTTP dates

LEVEL 5: VERY COMPLEX

5.1 Long-running jobs: `202 Accepted` + polling (VERIFIED)

Shape: `POST /jobs` → `202` + job id; `GET /jobs/{id}` → `queued | running(progress) | succeeded(result_url) | failed(error)`; `DELETE /jobs/{id}` cancels.

The use case returns **one stream**: start → status → status → … → terminal. Design decisions:

- polling **slows down** (1 s, 2 s, 4 s, capped at 5 s): a 5-minute job is ~70 requests, not 300;
- a **timeout** ends a stuck job and cancels it on the server;
- **cancelling the subscription** cancels the job server-side (`finally`);
- an **unknown** status string fails loudly (`ParsingFailure`) instead of polling forever;
- a terminal `failed` status is *data*, not an exception.

`feature/jobs/domain/usecase/run_job.dart`

```dart
import 'dart:async';

import '../../../../core/utils/backoff.dart';
import '../entity/job.dart';
import '../repository/job_repository.dart';

/// "Start it, then ask until it is done", as ONE stream:
///   start -> status -> status -> ... -> Succeeded | Failed | timeout
/// * polling slows down (backoff) so a 5-minute job is not 300 requests/s
/// * cancelling the SUBSCRIPTION cancels the job on the server too
/// * a stuck job ends with a timeout instead of polling forever
class RunJob {
  RunJob(
    this._repository, {
    this.firstInterval = const Duration(seconds: 1),
    this.maxInterval = const Duration(seconds: 5),
    this.timeout = const Duration(minutes: 2),
    Future<void> Function(Duration)? delay,
    DateTime Function()? now,
  }) : _delay = delay ?? Future<void>.delayed,
       _now = now ?? DateTime.now;

  final JobRepository _repository;
  final Duration firstInterval;
  final Duration maxInterval;
  final Duration timeout;
  final Future<void> Function(Duration) _delay;
  final DateTime Function() _now;

  Stream<JobStatus> call(JobRequest request) async* {
    final jobId = await _repository.start(request); // failures end the stream
    final startedAt = _now();
    var finished = false;
    try {
      for (var attempt = 1; ; attempt++) {
        final status = await _repository.status(jobId);
        yield status;
        if (status.isTerminal) {
          finished = true;
          return;
        }
        if (_now().difference(startedAt) >= timeout) {
          yield const JobFailed(
            'The job is taking too long. Please try again later.',
          );
          return; // `finished` stays false -> the finally block cancels it
        }
        await _delay(
          backoffDelay(
            attempt,
            base: firstInterval,
```

```
            max: maxInterval,
            jitter: 0,
          ),
        );
      }
    } finally {
      // Reached on: subscriber cancelled, timeout, or an error while polling.
      if (!finished) {
        try {
          await _repository.cancel(jobId);
        } catch (_) {
          // best effort
        }
      }
    }
  }
}
```

`feature/jobs/data/model/job_status_model.dart`

```
import '../../domain/entity/job.dart';

class JobStatusModel {
  const JobStatusModel({
    required this.status,
    this.progress,
    this.resultUrl,
    this.error,
  });

  final String status;
  final double? progress;
  final String? resultUrl;
  final String? error;

  factory JobStatusModel.fromJson(Map<String, dynamic> json) => JobStatusModel(
    status: json['status'] as String,
    progress: (json['progress'] as num?)?.toDouble(),
    resultUrl: json['result_url'] as String?,
    error: json['error'] as String?,
  );

  /// An unknown status is a contract break: fail loudly instead of polling a
  /// job whose state we cannot understand.
  JobStatus toEntity() => switch (status) {
    'queued' => const JobQueued(),
    'running' => JobRunning(((progress ?? 0) / 100).clamp(0, 1).toDouble()),
    'succeeded' => JobSucceeded(resultUrl!),
    'failed' => JobFailed(error ?? 'The job failed.'),
    _ => throw FormatException('unknown job status "$status"'),
  };
}
```

`feature/jobs/presentation/cubit/job_cubit.dart`

```
import 'dart:async';

import 'package:equatable/equatable.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../../../../core/errors/failures.dart';
import '../../../../core/errors/report_failure.dart';
import '../../domain/entity/job.dart';
import '../../domain/usecase/run_job.dart';

sealed class JobState extends Equatable {
  const JobState();

  @override
  List<Object?> get props => [];
```

```
}

class JobIdle extends JobState {
  const JobIdle();
}

class JobInProgress extends JobState {
  const JobInProgress(this.status);

  final JobStatus status; // Queued or Running(progress)

  @override
  List<Object?> get props => [status];
}

class JobDone extends JobState {
  const JobDone(this.resultUrl);

  final String resultUrl;

  @override
  List<Object?> get props => [resultUrl];
}

class JobError extends JobState {
  const JobError(this.message);

  final String message;

  @override
  List<Object?> get props => [message];
}

class JobCubit extends Cubit<JobState> with FailureReporting<JobState> {
  JobCubit(this._runJob) : super(const JobIdle());

  final RunJob _runJob;
  StreamSubscription<JobStatus>? _subscription;

  /// Completes when the job reaches a terminal state, fails, or is cancelled.
  Future<void> run(JobRequest request) async {
    if (state is JobInProgress) return;
    emit(const JobInProgress(JobQueued()));

    final done = Completer<void>();
    _subscription = _runJob(request).listen(
      (status) {
        if (isClosed) return;
        emit(switch (status) {
          JobSucceeded(:final resultUrl) => JobDone(resultUrl),
          JobFailed(:final reason) => JobError(reason),
          JobQueued() || JobRunning() => JobInProgress(status),
        });
      },
      onError: (Object error, StackTrace stack) {
        if (error is Failure) {
          reportFailure(error);
          if (!isClosed) emit(JobError(error.message));
        } else {
          addError(error, stack);
          if (!isClosed) {
            emit(const JobError('Something went wrong. Please try again.'));
          }
        }
      },
      onDone: () {
        _subscription = null;
        if (!done.isCompleted) done.complete();
      },
    );
    return done.future;
  }
```

```
  /// Cancelling the subscription makes RunJob cancel the job on the server.
  Future<void> cancel() async {
    final subscription = _subscription;
    if (subscription == null) return;
    _subscription = null;
    await subscription.cancel();
    if (!isClosed) emit(const JobIdle());
  }

  @override
  Future<void> close() async {
    await _subscription?.cancel();
    return super.close();
  }
}
```

16 tests cover: the status mapping, backoff sequence `[1,2,4,5,5]`, timeout + remote cancel, start failing, polling error, subscription cancel, and the Cubit's Done/Error/Cancel paths. (This pattern is exercised against a scripted server, because no public API offers jobs.)

5.2 Live streams: SSE, NDJSON, WebSocket (code compiles; UNTESTED)

> **Status:** this code passes the strict analyzer, but the test file for it was **not completed or run**. Treat it as a reviewed design, not a verified implementation.

- **SSE** (`text/event-stream`): fields `event/data/id/retry` , comments are keep-alives, **a blank line dispatches the event**, chunks may split a line or a UTF-8 character. On reconnect send `Last-Event-ID` to resume.
- **NDJSON**: one JSON object per line.
- **WebSocket**: each `connect()` opens a new connection, so it plugs straight into the reconnect helper.
- **Reconnect with backoff**: `Connecting → Message… → Reconnecting(1, 0.5 s) → Connecting(1)…` ; failures reset to 0 on every message; after N consecutive failures → `Closed` . A `4xx` (except 408/429) is **not** retried (retrying a 401 cannot fix it). Only `Exception` s are retried, so a bug surfaces instead of looping.
- **Bloc** is the right tool for a live screen: **one** handler for `Start` / `Stop` with `restartable()` means a new event cancels the previous `emit.forEach` (and so the connection).

`core/realtime/reconnecting_stream.dart`

```
import 'dart:math';

import '../errors/exceptions.dart';
import '../utils/backoff.dart';
import 'connection_event.dart';

/// 4xx (except 408/429) will fail the same way again: don't hammer the server.
bool defaultRetryable(Object error) {
  if (error is ServerException) {
    final code = error.statusCode;
    if (code != null &&
        code >= 400 &&
        code < 500 &&
        code != 408 &&
        code != 429) {
      return false;
    }
  }
  return true;
}

/// Keeps a stream alive. [connect] is called once per connection attempt.
///
///   Connecting(0) -> Message... -> (drop) -> Reconnecting(1, 0.5s)
///   -> Connecting(1) -> Message... (failures reset to 0 on every message)
///   ... after [maxFailures] consecutive failures -> Closed(lastError)
```

```dart
///
/// Only `Exception`s are retried: a bug (`Error`) surfaces instead of looping.
/// Cancelling the subscription cancels the live connection.
Stream<ConnectionEvent<T>> reconnecting<T>(
  Stream<T> Function() connect, {
  int maxFailures = 5,
  Duration base = const Duration(milliseconds: 500),
  Duration max = const Duration(seconds: 30),
  bool Function(Object error)? retryable,
  Future<void> Function(Duration)? delay,
  Random? random,
}) async* {
  final wait = delay ?? Future<void>.delayed;
  final canRetry = retryable ?? defaultRetryable;
  var failures = 0;
  Object? lastError;

  while (true) {
    yield Connecting<T>(failures);
    try {
      await for (final value in connect()) {
        failures = 0; // healthy again
        lastError = null;
        yield Message<T>(value);
      }
      // The server closed the stream normally: reconnect, with backoff.
    } on Exception catch (error) {
      lastError = error;
      if (!canRetry(error)) {
        yield Closed<T>(error);
        return;
      }
    }

    failures++;
    if (failures > maxFailures) {
      yield Closed<T>(lastError);
      return;
    }
    final pause = backoffDelay(failures, base: base, max: max, random: random);
    yield Reconnecting<T>(failures, pause);
    await wait(pause);
  }
}
```

`core/realtime/sse.dart`

```dart
import 'dart:convert';

import 'package:equatable/equatable.dart';

import '../network/dio_client.dart';
import 'connection_event.dart';
import 'reconnecting_stream.dart';

/// One Server-Sent Event (text/event-stream).
class ServerEvent extends Equatable {
  const ServerEvent({
    this.event = 'message',
    required this.data,
    this.id,
    this.retry,
  });

  final String event;
  final String data;
  final String? id;
  final int? retry; // server's suggested reconnect delay, ms

  @override
  List<Object?> get props => [event, data, id, retry];
```

```
}

/// Bytes -> events, per the SSE spec:
///  * fields: `event:` `data:` `id:` `retry:`; lines starting with `:` are comments
///  * several `data:` lines are joined with "\n"
///  * a BLANK line dispatches the event; an unfinished event at EOF is dropped
///  * chunks may split a line, or even a multi-byte UTF-8 character
Stream<ServerEvent> parseSse(Stream<List<int>> bytes) async* {
  String? event;
  String? id;
  int? retry;
  final data = <String>[];

  await for (final line
      in bytes.transform(utf8.decoder).transform(const LineSplitter())) {
    if (line.isEmpty) {
      if (data.isNotEmpty) {
        yield ServerEvent(
          event: event ?? 'message',
          data: data.join('\n'),
          id: id,
          retry: retry,
        );
      }
      event = null;
      data.clear();
      continue;
    }
    if (line.startsWith(':')) continue; // comment / keep-alive

    final colon = line.indexOf(':');
    final field = colon < 0 ? line : line.substring(0, colon);
    var value = colon < 0 ? '' : line.substring(colon + 1);
    if (value.startsWith(' ')) value = value.substring(1);

    switch (field) {
      case 'event':
        event = value;
      case 'data':
        data.add(value);
      case 'id':
        id = value; // "last event id" persists until changed
      case 'retry':
        retry = int.tryParse(value);
    }
  }
}

/// Newline-delimited JSON: one JSON object per line (many streaming APIs).
Stream<Map<String, dynamic>> parseNdjson(Stream<List<int>> bytes) async* {
  await for (final line
      in bytes.transform(utf8.decoder).transform(const LineSplitter())) {
    if (line.trim().isEmpty) continue;
    yield jsonDecode(line) as Map<String, dynamic>;
  }
}

class SseClient {
  const SseClient(this._client);

  final DioClient _client;

  /// One connection. [lastEventId] asks the server to RESUME after that event.
  Stream<ServerEvent> events(String url, {String? lastEventId}) async* {
    final bytes = await _client.openStream(
      url,
      headers: {
        'Accept': 'text/event-stream',
        'Cache-Control': 'no-cache',
        'Last-Event-ID': ?lastEventId,
      },
    );
```

```
    yield* parseSse(bytes);
  }

  /// Auto-reconnect that RESUMES where it left off (Last-Event-ID).
  Stream<ConnectionEvent<ServerEvent>> stream(
    String url, {
    int maxFailures = 5,
    Future<void> Function(Duration)? delay,
  }) {
    String? lastId;
    return reconnecting<ServerEvent>(
      () => events(url, lastEventId: lastId).map((event) {
        if (event.id != null) lastId = event.id;
        return event;
      }),
      maxFailures: maxFailures,
      delay: delay,
    );
  }
}
```

`feature/live/presentation/bloc/live_feed_bloc.dart`

```
import 'package:bloc_concurrency/bloc_concurrency.dart';
import 'package:equatable/equatable.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../../../../core/realtime/connection_event.dart';

sealed class LiveEvent extends Equatable {
  const LiveEvent();

  @override
  List<Object?> get props => [];
}

class LiveStarted extends LiveEvent {
  const LiveStarted();
}

class LiveStopped extends LiveEvent {
  const LiveStopped();
}

enum LiveStatus { idle, connecting, live, reconnecting, closed }

class LiveState extends Equatable {
  const LiveState({
    this.status = LiveStatus.idle,
    this.messages = const [],
    this.attempt = 0,
  });

  final LiveStatus status;
  final List<String> messages;
  final int attempt;

  LiveState copyWith({
    LiveStatus? status,
    List<String>? messages,
    int? attempt,
  }) => LiveState(
    status: status ?? this.status,
    messages: messages ?? this.messages,
    attempt: attempt ?? this.attempt,
  );

  @override
  List<Object?> get props => [status, messages, attempt];
}
```

```
/// A stream as a screen: Bloc is the right tool here (events in, a stream of
/// states out, and a transformer to cancel the previous run).
class LiveFeedBloc extends Bloc<LiveEvent, LiveState> {
  LiveFeedBloc(this._connect, {this.keep = 50}) : super(const LiveState()) {
    // ONE handler for both events + restartable(): Stop (or a second Start)
    // cancels the running `emit.forEach`, which cancels the connection.
    on<LiveEvent>(_onEvent, transformer: restartable());
  }

  final Stream<ConnectionEvent<String>> Function() _connect;
  final int keep;

  Future<void> _onEvent(LiveEvent event, Emitter<LiveState> emit) async {
    switch (event) {
      case LiveStopped():
        emit(state.copyWith(status: LiveStatus.idle));
      case LiveStarted():
        emit(const LiveState());
        await emit.forEach<ConnectionEvent<String>>(
          _connect(),
          onData: _reduce,
          onError: (error, stack) {
            addError(error, stack);
            return state.copyWith(status: LiveStatus.closed);
          },
        );
    }
  }

  LiveState _reduce(ConnectionEvent<String> event) => switch (event) {
    Connecting<String>(:final attempt) => state.copyWith(
      status: attempt == 0 ? LiveStatus.connecting : LiveStatus.reconnecting,
    ),
    Message<String>(:final value) => state.copyWith(
      status: LiveStatus.live,
      attempt: 0,
      messages: _capped([...state.messages, value]),
    ),
    Reconnecting<String>(:final attempt) => state.copyWith(
      status: LiveStatus.reconnecting,
      attempt: attempt,
    ),
    Closed<String>() => state.copyWith(status: LiveStatus.closed),
  };

  List<String> _capped(List<String> all) =>
      all.length > keep ? all.sublist(all.length - keep) : all;
}
```

5.3 Design recipes (NOT built or tested in this book)

These are the patterns you will be asked about. Each is a **design**, not verified code.

Offline-first with an outbox.

- The local database is the **source of truth**; the UI reads it (a stream).
- A write goes to the DB **and** to an *outbox* table (`id` = idempotency key, `type` , `payload` , `attempts`).
- A syncer sends the outbox **in order**: success → delete; **permanent** failure (validation, 4xx except 408/409/429) → dead-letter + tell the user; **transient** (offline, 5xx, 429) → `attempts++` , stop (keep order), retry with backoff when connectivity returns.
- Send the outbox id as an `Idempotency-Key` (the posts data source already supports the header) so a retry after a timeout cannot duplicate the write.
- Decide a **conflict policy** up front: server wins / last write wins / merge / ask the user.

OAuth2 Authorization Code + PKCE.

- Generate a random `code_verifier` (43–128 chars); `code_challenge = base64url(SHA-256(verifier))` , method `S256` .

- Open the system browser (`flutter_appauth` / `url_launcher`) with `state` (anti-CSRF) and the challenge; verify `state` on the redirect.
- Exchange `code + code_verifier` at the token endpoint (`application/x-www-form-urlencoded`). Keep tokens in secure storage; refresh as in the first book (§12–13).
- A test vector exists in RFC 7636 Appendix B: use it when you implement it.

Signed requests (HMAC).

- An interceptor adds `X-Timestamp` , `X-Nonce` and `X-Signature = HMAC-SHA256(secret, method + path&query + timestamp + nonce + SHA256(body))` .
- Sign the **exact bytes you send** (serialise JSON yourself). Remember: a secret shipped in an app can be extracted: signing gives tamper-evidence, not authentication.

Environments and API versions.

- One `AppConfig` per environment (`dev/staging/prod`) from `--dart-define=ENV=staging` ; fail fast on an unknown value.
- Version via URL (`/v2`), header, or media type. Keep **one model per version mapping to the same entity** so the domain doesn't change.

Large JSON. Parse hundreds of KB off the UI thread with `Isolate.run(() => parse(raw))` (the parse function must be top-level/static). Measure first: small lists don't need it.

PART B: Cheat sheets

B.1 "I see X in the API → do Y" matrix

I see…	Do this	Where	Level
Bare array response	`(json as List)`, add **Empty** state	model + state	1
`{"data": ..., "total": ...}` envelope	Response model → entity; keep `total` only if used	model	1–2
Detail endpoint	one `get`, bare/wrapped object	data source	2
Filters / search	query params, omit empty, reload page 1	data source + cubit	2
Writes	request model (`toJson`), `Command` Cubit	model + cubit	2
Field validation errors (422)	`ProblemDetails` → `ValidationFailure.fieldErrors`	core	2–4
`x-total-count` / `Link` headers	`getResponse` + `parseLinkHeader` → `NextUrl`	data source	3
Cursor in body	`CursorToken(next_cursor)`	data source	3
Related resources	parallel + dependent calls in a use case; optional sections	use case	3
File upload	`FormData` + progress + `UploadTask` + cancel	data + domain	3
`type` discriminator	sealed class + `UnknownItem` + tolerant list	model	4
`429` + `Retry-After`	`RetryAfterInterceptor`	core	4
`ETag` / `304`	`ETagCacheInterceptor`	core	4
Want instant screens	stale-while-revalidate	use case	4
GraphQL	`GraphQLClient`; **check `errors` on HTTP 200**	core	4
`202 Accepted` + status URL	poll with backoff, timeout, cancel	use case	5
SSE / WebSocket	reconnecting stream + Bloc (untested here)	core + bloc	5
Offline writes	outbox + idempotency key (design only)	repository	5
OAuth2 / signed requests	recipes in 5.3 (design only)	core	5

B.2 What to build in 20 minutes at each level

Level	Build (LITE)	Say aloud, don't build
1	model + entity, repo interface + impl, `GetX`, Cubit with Empty/Error/Loaded, page	`guard` / `Failure` hierarchy, pagination
2	+ detail + filter + one write (`POST`) with a Command Cubit	optimistic delete, `PATCH` null-aware map
3	+ `PaginatedCubit`-style `loadMore` with the **Link/total from headers**	upload progress, parallel use case
4	one hard thing the interviewer names (polymorphism **or** GraphQL **or** caching)	the other three, as designs
5	explain only: stream result, backoff, idempotency, PKCE	everything

B.3 Universal rules (all levels)

1. **Layers never change**: only the model shape, the token, the interceptor and the state shape do.
2. **One error vocabulary**: `DioException → AppException → Failure → state`; user-safe messages; technical cause to the logger.

3. **Parse strictly, degrade deliberately**: strict required fields, nullable optionals, `Unknown*` for new server values.
4. **Never trust HTTP 200** (GraphQL) or an empty body (204).
5. **Anything that can repeat needs a guard**: duplicate `loadMore`, double submit, parallel 401, retried `POST` (idempotency key).
6. **Everything long-lived needs a cancel path**: upload, job, stream, subscription.
7. **Test the boundary with a scripted HTTP layer** (a fake adapter), not the network.

PART C: Interview questions for "integrate this API" (concise answers)

1. What do you look at first in an unfamiliar API? Real responses: list shape (bare vs envelope), where pagination lives, nullability, error format, auth, rate limits. The recon table decides which layer changes.

2. The API returns a bare list. What changes? The model parses the root list; there is no `data` key; I add an Empty state; pagination info probably lives in headers.

3. How do you paginate with `Link` headers? Parse `rel="next"` and follow that URL verbatim (filters are inside it). `NextUrl` is a `PageToken`; the same generic Cubit handles page-number, cursor and next-URL APIs.

4. How do you avoid duplicate `loadMore` calls? Guard on state type, `isLoadingMore`, and last page; a generation counter drops stale responses; the footer triggers when built.

5. PUT vs PATCH? PUT replaces the whole resource (all fields); PATCH sends only changed fields. In Dart: `{'title': ?title}` skips nulls.

6. How do you show server-side validation errors? Parse problem+json / `errors` into `fieldErrors`, throw `ValidationFailure`; the form shows messages under each field. Client-side rules use the *same* shape, so one code path.

7. How do you make a `POST` safe to retry? Send an `Idempotency-Key` (a stable id per user action); the server dedupes. I never blindly retry a POST without one.

8. What is optimistic UI, and its risk? Update the UI before the server answers. Risk: the server refuses. So I restore the previous state at the same position and inform the user.

9. A screen needs three endpoints. How? Independent calls start together; dependent ones wait; required vs optional sections; one record `.wait`; a failing optional section degrades instead of failing the screen.

10. How do you handle uploads? Multipart with throttled progress; the domain sees an `UploadTask` (progress/result/cancel); closing the screen cancels it.

11. What's special about GraphQL errors? HTTP 200 with an `errors` array. I check the body, map `extensions.code` to HTTP-like statuses, and treat `data: null` for a missing id as "not found", not an error.

12. How do you handle `429`? Honour `Retry-After` (seconds or date), capped, retry a limited number of times via a plain Dio, then surface `RateLimitedFailure(retryAfter)`.

13. What is ETag caching? Conditional GET: send `If-None-Match`; `304` means reuse my cached body. Saves bandwidth; the caller never sees the 304.

14. What is stale-while-revalidate? Show cached data instantly marked stale, fetch fresh, replace. If the refresh fails, keep stale data and flag it. A broken cache must not break loading.

15. How do you parse polymorphic JSON? Switch on the `type` discriminator into a sealed hierarchy; keep unknown types as `UnknownItem`; skip and count malformed items instead of failing the whole list.

16. How do you poll a long job? Start (202), then poll status with backoff and a timeout; expose it as a Stream; cancelling the subscription cancels the job remotely; unknown status = loud failure.

17. SSE vs WebSocket? SSE: one-way server→client over HTTP, auto-resume with `Last-Event-ID`, simple. WebSocket: two-way. Both need reconnect with backoff and no retry on 4xx.

18. Why Bloc for a live screen? Events in, state stream out, and transformers: one `restartable()` handler lets Stop/Start cancel the running stream cleanly.

19. How would you do offline-first? DB as source of truth, outbox of pending writes with idempotency keys, ordered sync, permanent vs transient failures, a conflict policy.

20. How do you secure OAuth in a mobile app? Authorization Code with PKCE, `state` check, system browser, tokens in secure storage, refresh-token rotation, never an embedded WebView.

21. How do you test an HTTP layer without a network? A scripted `HttpClientAdapter`: the test decides every response, header and delay; add real local servers (WebSocket) where it's cheap.

22. How do you support API versions? Version in URL/header; one model per version mapping to the same entity; migrate data sources one at a time behind a flag.

23. What do you do when you can't tell if an endpoint is idempotent? Treat it as non-idempotent: no automatic retry, add an idempotency key if the API supports it.

24. What would you NOT build in a 20-minute round? Interceptors, caching, GraphQL/streams/offline: I name them, describe the design and the files they'd go in, and build the core flow cleanly.

PART D: Verification record and known limits

Check	Result
`flutter analyze` (strict-casts / strict-inference / strict-raw-types), library	**0 issues**
Tests: core (helpers, `DioClient`, interceptors, `PaginatedCubit`), posts (data, use cases, cubits, pages), files, level 4 (SWR, GraphQL, feed), jobs	**156 passed**
Live: JSONPlaceholder (list, pagination to the last page, filter, search, empty, details, POST/PUT/PATCH/DELETE, 404, ETag 304)	✅ outputs shown in Levels 1–4
Live: httpbin (300 KB multipart upload with progress, cancel, download)	✅
Live: countries GraphQL (query, null result, list, HTTP-200 error)	✅
Real-time stack (SSE / WebSocket / reconnect / `LiveFeedBloc`)	Analyzer-clean; **tests not run**
Outbox, PKCE, signing, environments, isolate parsing	**Not built**: design recipes only

Bugs the tests caught while building this book (and are fixed): a generic Cubit emitting `PaginatedLoading<Never>` instead of `<T>` (state equality failed); an upload whose `result` would hang when nobody listened to progress; plus several test-fixture mistakes. This is why the code was tested, not just compiled.

Not covered: device/emulator runs, performance measurements, real SSE/WebSocket servers, authentication against a real OAuth provider.